HHL

The Quantum Linear-Systems Algorithm

A step-by-step chapter: from $Ax = b$ to a CUDA-Q implementation

What you will be able to do after this chapter

- solve small linear systems through the eigen-decomposition and compute condition numbers, sparsity and classical costs;
- amplitude-encode vectors, evaluate e^{iAt} and explain why HHL needs Hermitian, invertible, well-conditioned matrices;
- run quantum phase estimation by hand, including inexact phases and two's-complement (negative) clock values;
- derive every state of the HHL algorithm: eigenvalue inversion, the constant C, the ancilla, uncomputation and post-selection;
- compute success probabilities, read-out statistics and the norm of x, and judge the complexity claims honestly;
- read and modify `HHL.ipynb`, and interpret its circuit statistics and results;
- solve every type of question in the accompanying 80-question HHL assessment.

SERGIO FELLER

Companion material in the same repository: `presentations/HHL_Lecture.pdf` (slides), `HHL.ipynb` (CUDA-Q notebook) and `assessments/` (question paper and answer book).

Contents

HHL — The Quantum Linear-Systems Algorithm **4**

1 Linear systems and the linear algebra behind HHL 5

1.1 Linear systems 5

1.2 Hermitian matrices and the eigen-decomposition 5

1.3 Non-Hermitian matrices: the embedding 6

1.4 The condition number 6

1.5 Sparsity and classical cost 7

2 From vectors to quantum states 7

2.1 Amplitude encoding 7

2.2 Unitary time evolution e^{iAt} 7

2.3 Memory of a simulated state 8

3 Phase estimation: the engine of HHL 8

3.1 Controlled unitaries and phase kick-back 8

3.2 The quantum Fourier transform 8

3.3 Quantum phase estimation 9

3.4 Phase estimation with $U = e^{iAt}$: eigenvalues in the clock 9

4 The HHL algorithm, step by step 10

4.1 Registers 10

4.2 The circuit 10

4.3 The state after every stage 11

4.4 Eigenvalue inversion and the constant C 11

4.5 The ancilla and uncomputation 11

4.6 Success probability and repetitions 12

5 Fully worked problems 13

6 Reading out the solution 15

7 Complexity and its fine print 16

8 The implementation in `HHL.ipynb` 17

8.1 Structure and parameters 17

8.2 The circuit code 17

8.3 Circuit size 18

SILICO FELLER

8.4 Results 18
9 Noise, limitations and advanced algorithms 19
9.1 Errors 19
9.2 Limitations 19
9.3 Applications and newer algorithms 19
10 Problem-solving strategies and question map 20
10.1 Strategy by question format 20
10.2 Fast checks that catch most mistakes 20
10.3 Map: assessment questions → sections of this chapter 20
11 Practice problems 21

SILICO FELLER

HHL — The Quantum Linear-Systems Algorithm

How to use this chapter

The chapter is built as a staircase. Sections 1–2 are the *linear algebra* you need (linear systems, eigen-decomposition, Hermitian matrices, condition number, sparsity) and how vectors become *quantum states*. Section 3 is the *quantum toolkit* (controlled unitaries, the QFT and phase estimation). Sections 4–5 are *HHL itself*: the algorithm derived line by line and fully worked examples. Sections 6–7 cover *what you can read out* and *what it costs*. Sections 8–9 cover the *implementation* in `HHL.ipynb`, its results, noise, limitations and newer algorithms. Section 10 collects problem-solving strategies and maps every assessment question to the section that teaches it; Section 11 has practice problems with answers.

Each section has: the idea in words, the precise statement, a derivation, **worked examples** solved step by step, **common traps** (the wrong answers examiners like to offer) and a short checklist. Statements are marked as *general theory*, *the notebook's implementation*, or *illustrative examples*. "Recorded" results are the outputs stored in `HHL.ipynb`; "re-executed" results were produced again with CUDA-Q 0.16 (`qpp-cpu`) from the notebook's circuit code. Every number was recomputed with NumPy or CUDA-Q (`assessments/hhl_test_source/verify_answers.py`, `presentations/hhl_source/make_figures.py`).

Definition: Conventions used everywhere in this chapter

- A is $d \times d$ with $d = 2^{n_b}$; eigenpairs $A\,|u_j\rangle = \lambda_j\,|u_j\rangle$; $|b\rangle = \sum_j \beta_j\,|u_j\rangle$ with $\beta_j = \langle u_j|b\rangle$.
- $U = e^{iAt}$ (the notebook's sign), n_c clock qubits, $M = 2^{n_c}$, $C = 2\pi/(Mt)$. Clock value of λ: $k = \lambda t M/(2\pi) = \lambda/C$.
- Clock register: `clock[0]` is the **most significant bit**; values $k \geq M/2$ are negative: $\tilde{k} = k - M$ (two's complement).
- $R_Y(\theta) = e^{-i\theta Y/2}$, so $R_Y(\theta)\,|0\rangle = \cos\frac{\theta}{2}\,|0\rangle + \sin\frac{\theta}{2}\,|1\rangle$.
- $\mathrm{QFT}\,|c\rangle = \frac{1}{\sqrt{M}}\sum_k e^{2\pi i\,ck/M}\,|k\rangle$.
- The notebook's default system: $A = H\,\mathrm{diag}(1,2,3,4)\,H$ with $H = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}$, $b = (1,0,0,0)$, $n_c = 4$, $t = \pi/8$, $C = 1$, 7 qubits, 20 000 shots, seed 42.

1 Linear systems and the linear algebra behind HHL

1.1 Linear systems

A linear system $Ax = b$ has a unique solution $x = A^{-1}b$ exactly when A is **invertible**: $\det A \neq 0$, equivalently no eigenvalue is 0.

Worked example 1.1: a 2×2 system by elimination

$A = \left(\begin{smallmatrix} 1 & -1/3 \\ -1/3 & 1 \end{smallmatrix}\right)$, $b = (1, 0)$ (the notebook's 2×2 example).

Step 1. $\det A = 1 - \frac{1}{9} = \frac{8}{9} \neq 0$.

Step 2. $A^{-1} = \frac{1}{\det A}\left(\begin{smallmatrix} 1 & 1/3 \\ 1/3 & 1 \end{smallmatrix}\right) = \frac{9}{8}\left(\begin{smallmatrix} 1 & 1/3 \\ 1/3 & 1 \end{smallmatrix}\right)$.

Step 3. $x = A^{-1}b = \frac{9}{8}(1, \frac{1}{3}) = (\frac{9}{8}, \frac{3}{8}) = (1.125, 0.375)$. Check: $1.125 - 0.375/3 = 1$ and $-1.125/3 + 0.375 = 0$ ✓.

1.2 Hermitian matrices and the eigen-decomposition

Key result: spectral theorem

A Hermitian matrix ($A = A^\dagger$; real symmetric if real) has real eigenvalues and an orthonormal eigenbasis: $A = \sum_j \lambda_j |u_j\rangle \langle u_j|$. If no λ_j is 0,

$$A^{-1} = \sum_j \frac{1}{\lambda_j} |u_j\rangle \langle u_j|, \qquad A^{-1}|b\rangle = \sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle.$$

Inverting A means *dividing each eigen-component of b by its eigenvalue*. HHL does precisely this, for all components at once, in superposition. Other classes: unitary matrices have $|\lambda| = 1$; positive-definite matrices have $\lambda > 0$; projectors have $\lambda \in \{0, 1\}$ (and are singular unless $P = I$).

Worked example 1.2: solving through the eigenbasis

$A = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$, $b = (2, 1)$.

Step 1. $\lambda_1 = 3$, $|u_1\rangle = \frac{1}{\sqrt{2}}(1, 1)$; $\lambda_2 = 1$, $|u_2\rangle = \frac{1}{\sqrt{2}}(1, -1)$.

Step 2. $\beta_1 = \frac{2+1}{\sqrt{2}} = \frac{3}{\sqrt{2}}$, $\beta_2 = \frac{2-1}{\sqrt{2}} = \frac{1}{\sqrt{2}}$.

Step 3. $x = \frac{3/\sqrt{2}}{3}|u_1\rangle + \frac{1/\sqrt{2}}{1}|u_2\rangle = \frac{1}{2}(1, 1) + \frac{1}{2}(1, -1) = (1, 0)$. Check: $A(1, 0) = (2, 1)$ ✓.

Worked example 1.3: the notebook's default matrix

H is symmetric and orthogonal ($H^2 = I$), so $A = H\,\mathrm{diag}(1, 2, 3, 4)\,H$ has eigenvalues $1, 2, 3, 4$ and eigenvectors = the columns of H. Writing it out,

$$A = \begin{pmatrix} 2.5 & -0.5 & -1 & 0 \\ -0.5 & 2.5 & 0 & -1 \\ -1 & 0 & 2.5 & -0.5 \\ 0 & -1 & -0.5 & 2.5 \end{pmatrix}.$$

Check: trace $= 10 = 1 + 2 + 3 + 4$; each row has 3 non-zeros.

1.3 Non-Hermitian matrices: the embedding

Phase estimation reads eigenvalues of the *unitary* e^{iAt}, which is unitary for every t only if A is Hermitian. A general A is embedded:

$$\tilde{A} = \begin{pmatrix} 0 & A \\ A^\dagger & 0 \end{pmatrix}, \qquad \tilde{A}\begin{pmatrix} y \\ x \end{pmatrix} = \begin{pmatrix} b \\ 0 \end{pmatrix} \iff Ax = b,\ A^\dagger y = 0\ (\Rightarrow y = 0).$$

The solution sits in the **second half**; the dimension doubles (one more system qubit). The eigenvalues of $\tilde{A}$ are $\pm\sigma_i$, the singular values of A with both signs — so the embedded system always has **negative eigenvalues**. Dimensions that are not powers of 2 are padded with an identity block.

Worked example 1.4: embedding $\left(\begin{smallmatrix} 1 & 2 \\ 0 & 1 \end{smallmatrix}\right)$ (notebook, Section 4 table)

Step 1. $A^\dagger A = \left(\begin{smallmatrix} 1 & 2 \\ 2 & 5 \end{smallmatrix}\right)$ has eigenvalues $3 \pm 2\sqrt{2}$, so $\sigma = \sqrt{2} \pm 1 = 2.4142, 0.4142$; $\kappa = (\sqrt{2}+1)^2 = 5.83$.

Step 2. $\tilde{A}$ is 4×4 with eigenvalues ± 2.4142, ± 0.4142; $n_b = 2$, so with $n_c = 4$ the circuit has $4 + 2 + 1 = 7$ qubits.

Step 3. $\kappa = 5.83 \leq 7$: "auto" time maps 0.4142 to clock value 1, $C = 0.4142$; then 2.4142 maps to 5.83 — *not an integer*.

Step 4. Simulating the notebook's circuit (NumPy and CUDA-Q agree): $P_{\text{success}} = 0.1696$, recovered second half $(-1.0085, 0.9796)$ vs. the exact $x = (-1, 1)$, fidelity 0.99979. The rounding of 5.83 costs a little accuracy.

Common trap

"The embedded system's solution is the first half" — no: (y, x) with x in the second half. And "padding with an identity block is harmless" — it adds the eigenvalue 1: a 3×3 matrix with eigenvalues $5, 10, 20$ ($\kappa = 4$) becomes 4×4 with $\kappa = 20/1 = 20$ for the clock.

1.4 The condition number

Definition: condition number

$\kappa(A) = \|A\|\,\|A^{-1}\| = \sigma_{\max}/\sigma_{\min} \geq 1$; for Hermitian A, $\kappa = |\lambda|_{\max}/|\lambda|_{\min}$. It is unchanged by $A \to cA$ ($c \neq 0$).

κ is the worst-case amplification of relative errors: $\frac{\|\delta x\|}{\|x\|} \leq \kappa \frac{\|\delta b\|}{\|b\|}$.

Worked example 1.5: error amplification

$A = \text{diag}(1, 0.01)$ ($\kappa = 100$), $b = (1, 0)$, $x = (1, 0)$. Perturb b by $\delta b = (0, 0.01)$ (relative size 0.01): $\delta x = (0, 1)$ (relative size 1). The error grew by exactly $100 = \kappa$.

For HHL, κ decides three things: how many clock qubits are needed to resolve $\lambda_{\min}$ next to $\lambda_{\max}$, how small the success probability can be (Section 4.6), and therefore the run time (Section 7).

Common trap

κ is a ratio of eigenvalue *magnitudes* — not $\lambda_{\max} - \lambda_{\min}$, not $\lambda_{\max}/\lambda_{\min}$ with signs. Unitary matrices have $\kappa = 1$.

1.5 Sparsity and classical cost

A matrix is s-**sparse** if every row has at most s non-zero entries (the default A: $s = 3$). Sparse matrices need $O(ds)$ storage and $O(ds)$ work per product Av.

method	cost	remark
Gaussian elimination / LU	$\frac{2}{3}d^3$ flops	dense; doubling d costs $8\times$
Conjugate gradient (positive definite)	$O(d\,s\sqrt{\kappa}\log(1/\varepsilon))$	iterations $\propto \sqrt{\kappa}$
CG on the normal equations (general A)	$O(d\,s\,\kappa\log(1/\varepsilon))$	squares the condition number
HHL (Section 7)	$\tilde{O}(\log(d)\,s^2\kappa^2/\varepsilon)$	returns $\lvert x\rangle$, not x

The notebook's reference solution is `np.linalg.solve` (LU). Any classical method that writes down x costs at least d.

Checklist

Invertible? Hermitian (or embedded, solution in the second half)? κ from eigenvalue *magnitudes*? Did padding add the eigenvalue 1?

2 From vectors to quantum states

2.1 Amplitude encoding

Definition: amplitude encoding

A vector $b \in \mathbb{C}^d$, $d = 2^n$, is stored in n qubits as $\lvert b\rangle = \frac{1}{\lVert b\rVert}\sum_i b_i \lvert i\rangle$. Only the direction is stored; $\lVert b\rVert$ is kept classically.

Worked example 2.6: encoding and measuring

$b = (3, 0, 4, 0)$: $\lVert b\rVert = 5$, $\lvert b\rangle = 0.6\lvert 00\rangle + 0.8\lvert 10\rangle$ (index $i = 2q_0 + q_1$). Measuring gives $\lvert 00\rangle$ with probability 0.36 and $\lvert 10\rangle$ with 0.64: the first qubit is $\lvert 1\rangle$ with probability 0.64, the second qubit is always $\lvert 0\rangle$.

Exponential compactness cuts both ways: n qubits hold 2^n amplitudes, but a generic $\lvert b\rangle$ needs $O(2^n)$ elementary gates to prepare (Möttönen, Shende–Bullock–Markov), and reading the amplitudes back needs many measurements. Efficient preparation needs structure in b or a QRAM. **In the notebook**, the whole initial state $\lvert 0\rangle_c \lvert b\rangle \lvert 0\rangle_a$ is loaded with `kernel.qalloc(vector)` — a simulator feature without gates; the reported gate counts do not include state preparation.

2.2 Unitary time evolution e^{iAt}

For Hermitian $A = \sum_j \lambda_j \lvert u_j\rangle\langle u_j\rvert$: $e^{iAt} = \sum_j e^{i\lambda_j t}\lvert u_j\rangle\langle u_j\rvert$ — same eigenvectors, eigenvalues on the unit circle, unitary for every real t (it is *not* unitary in general if A is not Hermitian).

Worked example 2.7: two exponentials

Step 1. $A = \mathrm{diag}(1, 2)$, $t = \pi/2$: $e^{iAt} = \mathrm{diag}(e^{i\pi/2}, e^{i\pi}) = \mathrm{diag}(i, -1)$.

Step 2. $A = X$ (Pauli): $X^2 = I$, so $e^{iXt} = \cos t\, I + i \sin t\, X$. At $t = \pi/2$: iX.

Hamiltonian simulation is the task of implementing e^{iAt} with gates. For $A = \sum_k A_k$ with simple pieces, the Trotter product $\left(\prod_k e^{iA_k t/r}\right)^r$ has error $O(t^2/r)$ (first order); sparse matrices are split into 1-sparse pieces; modern methods (qubitization, quantum signal processing) are near-optimal. **In the notebook** there is no Hamiltonian simulation: `U_step = expm(1j * A_work * t_evolution)` is the exact matrix exponential, registered as a custom gate.

2.3 Memory of a simulated state

A state vector of N qubits has 2^N complex amplitudes: $16 \cdot 2^N$ bytes in fp64, $8 \cdot 2^N$ in fp32; a density matrix needs $16 \cdot 4^N$. The default HHL circuit ($N = 7$) needs 2048 bytes; the notebook's memory line counts one extra copy for `get_state` (about 4.1 kB). With $n_c = 8$ ($N = 11$): 32 768 bytes per copy. Memory is never the bottleneck of these small instances; gate count and depth are.

Checklist

Normalised amplitudes? Probabilities = |amplitude|^2? Gate cost of state preparation counted (or explicitly excluded)?

3 Phase estimation: the engine of HHL

3.1 Controlled unitaries and phase kick-back

A controlled-V gate is $|0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes V$. If the target is an eigenvector, $V|u\rangle = e^{i\phi}|u\rangle$, then

$$\tfrac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\,|u\rangle \longmapsto \tfrac{1}{\sqrt{2}}(|0\rangle + e^{i\phi}|1\rangle)\,|u\rangle :$$

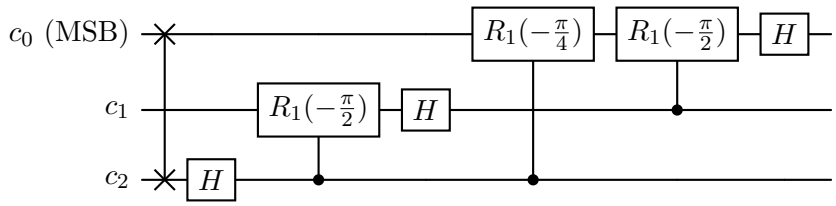
the phase is **kicked back** onto the control; the target is unchanged. A controlled-U^{2^j} can be built as 2^j controlled-U gates, or (cheaper) by simulating $e^{iA\,t2^j}$ directly; the notebook registers U^{2^j} = `matrix_power(U, 2**j)` as one custom gate.

3.2 The quantum Fourier transform

$\mathrm{QFT}|c\rangle = \frac{1}{\sqrt{M}}\sum_k e^{2\pi i\,ck/M}|k\rangle$ is unitary, so $\mathrm{QFT}^{-1} = \mathrm{QFT}^\dagger$ (reverse the gate order and negate the angles). Its inverse maps a **phase ramp** to a basis state:

$$\mathrm{QFT}^{-1} :\ \frac{1}{\sqrt{M}}\sum_{c=0}^{M-1} e^{2\pi i\,ck/M}|c\rangle \longmapsto |k\rangle\,.$$

Notebook implementation (n_c clock qubits): the inverse QFT is `swap`s first, then for $j = n_c - 1, \ldots, 0$ the controlled phases `cr1`$(-\pi/2^{k-j})$ on qubit j, controlled by each less significant qubit $k > j$, then `h` on qubit j. The QFT in the uncomputation is its exact mirror image (`h` and `cr1`$(+\pi/2^{k-j})$, then the swaps). For $n_c = 4$: 4 `h`, 6 `cr1`, 2 `swap` each.



The notebook's inverse QFT for $n_c = 3$, in code order.

3.3 Quantum phase estimation

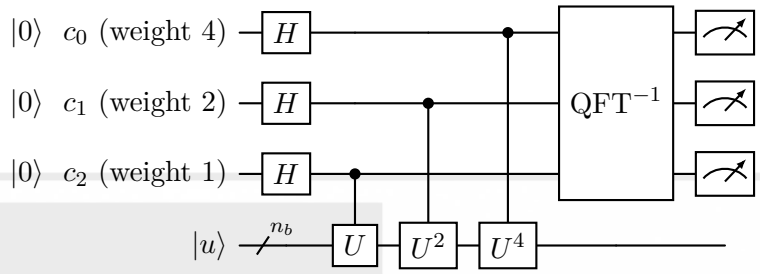
Key result: phase estimation

Let $U|u\rangle = e^{2\pi i\phi}|u\rangle$, $\phi \in [0,1)$. With n clock qubits, $M = 2^n$:

1. $H^{\otimes n}$: $\frac{1}{\sqrt{M}}\sum_c |c\rangle|u\rangle$;
2. controlled U^{2^j} by the clock qubit of weight 2^j (together: U^c on $|c\rangle$): $\frac{1}{\sqrt{M}}\sum_c e^{2\pi i\phi c}|c\rangle|u\rangle$;
3. QFT^{-1}: if $M\phi = k$ is an integer, the result is exactly $|k\rangle|u\rangle$. Otherwise outcome k has probability

$$P(k) = \Big|\frac{1}{M}\sum_{c=0}^{M-1} e^{2\pi i c(\phi - k/M)}\Big|^2,$$

and the nearest integer to $M\phi$ has $P \geq 4/\pi^2 \approx 0.405$.



Worked example 3.8: exact and inexact phases with two clock qubits

Step 1. $\phi = \frac{1}{4}$: after step 2 the clock holds $\frac{1}{2}(|0\rangle + i|1\rangle - |2\rangle - i|3\rangle)$ (phases $e^{2\pi ic/4} = 1, i, -1, -i$). This is $\mathrm{QFT}|1\rangle$, so QFT^{-1} gives $|01\rangle$ with probability 1: $\phi = 0.01_2$.

Step 2. $\phi = \frac{1}{3} = 0.0101\ldots_2$: $P(k) = (0.0625, 0.6998, 0.1875, 0.0502)$ for $k = 0,1,2,3$. The nearest $k = 1$ wins with ≈ 0.70; the rest **leaks**.

Step 3. $\phi = \frac{3}{16}$ with four clock qubits: $16\phi = 3$, outcome $|0011\rangle$ with certainty.

3.4 Phase estimation with $U = e^{iAt}$: eigenvalues in the clock

With $U = e^{iAt}$ the eigenphase of $|u_j\rangle$ is $\phi_j = \lambda_j t/(2\pi)$ and the clock reads

$$k_j = M\phi_j = \frac{\lambda_j\, t\, 2^{n_c}}{2\pi} = \frac{\lambda_j}{C}, \qquad C = \frac{2\pi}{2^{n_c} t}.$$

One clock step is worth C in eigenvalue units. Phases are taken modulo 1, so a **negative** eigenvalue $\lambda < 0$ appears as $k = M + \lambda/C$:

register k ($n_c = 3$)	0	1	2	3	4	5	6	7
signed $\tilde{k}$	0	1	2	3	−4	−3	−2	−1

For $n_c = 4$: $3 = \texttt{0011}$, $8 \to -8$, $12 \to -4$, $15 \to -1$. Consequences: the representable signed range is $-M/2, \ldots, M/2 - 1$, so we need $|\lambda|t < \pi$; an eigenvalue beyond the range **aliases** (with $n_c = 4$, $t = \pi/8$: $\lambda = 9$ is read as $9 - 16 = -7$). Shifting $A \to A + I$ shifts every clock value by $1/C$.

Worked example 3.9: the default system

$n_c = 4$, $t = \pi/8$: $k_j = \lambda_j \cdot \frac{(\pi/8)\cdot 16}{2\pi} = \lambda_j$. With $\beta_j = \frac{1}{2}$, after phase estimation

$$\tfrac{1}{2}\big(\,|u_1\rangle\,|0001\rangle + |u_2\rangle\,|0010\rangle + |u_3\rangle\,|0011\rangle + |u_4\rangle\,|0100\rangle\,\big)\,|0\rangle_a\,.$$

System and clock are now **entangled** (a product state only if $|b\rangle$ is a single eigenvector).

Common trap

Measuring the clock now would collapse the superposition onto one eigenvector: it estimates *one* eigenvalue, it does not solve the system. And $k_j = \lambda_j t/(2\pi)$ is the phase, not the clock value — multiply by 2^{n_c}.

Checklist

Which clock qubit is the MSB? Is $M\phi$ an integer? Negative eigenvalue $\Rightarrow k - M$? Is $|\lambda|t < \pi$?

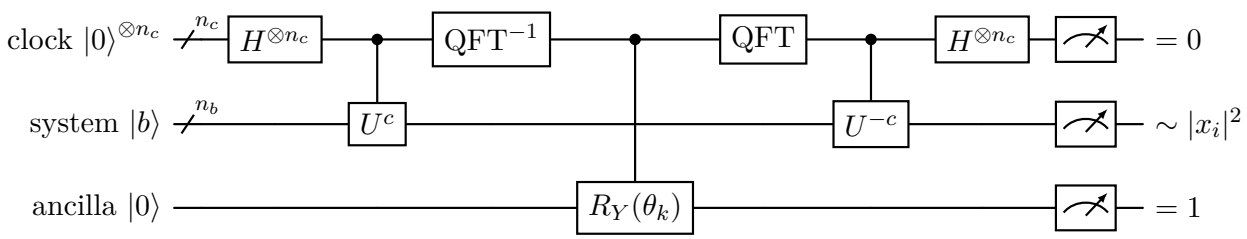
4 The HHL algorithm, step by step

4.1 Registers

register	qubits	role	default
clock	n_c	phase estimation: holds λ_j/C in binary	4 (qubits 0–3)
system	$n_b = \log_2 d$	starts in $\|b\rangle$, ends in $\|x\rangle$	2 (qubits 4–5)
ancilla	1	rotated by $C/\tilde{\lambda}$; outcome 1 = success	1 (qubit 6)
total	$n_c + n_b + 1$		7

Example: an 8×8 Hermitian system with $n_c = 5$ needs $5 + 3 + 1 = 9$ qubits.

4.2 The circuit



"Controlled U^c" denotes the n_c gates U^{2^j}, each controlled by one clock qubit; the inversion block is a set of rotations controlled on the clock value k.

4.3 The state after every stage

Key result: HHL state evolution (exact clock values)

0. initial	$\|0\rangle_c \|b\rangle \|0\rangle_a = \sum_j \beta_j \|0\rangle_c \|u_j\rangle \|0\rangle_a$
1–2. phase estimation	$\sum_j \beta_j \|k_j\rangle \|u_j\rangle \|0\rangle_a$
3. eigenvalue inversion	$\sum_j \beta_j \|k_j\rangle \|u_j\rangle \left(\sqrt{1-\frac{C^2}{\lambda_j^2}}\|0\rangle + \frac{C}{\lambda_j}\|1\rangle\right)$
4. uncomputation	$\sum_j \beta_j \|0\rangle_c \|u_j\rangle \left(\sqrt{1-\frac{C^2}{\lambda_j^2}}\|0\rangle + \frac{C}{\lambda_j}\|1\rangle\right)$
5. ancilla = 1	$\frac{C\sum_j \frac{\beta_j}{\lambda_j}\|u_j\rangle}{\sqrt{P_{\text{success}}}} = \|x\rangle\,, \qquad P_{\text{success}} = C^2 \sum_j \frac{\|\beta_j\|^2}{\lambda_j^2} = C^2 \,\|A^{-1}\|b\rangle\,\|^2.$

Derivation of step 5. The ancilla-$|1\rangle$ part of step 4 is $|0\rangle_c \otimes C\sum_j \frac{\beta_j}{\lambda_j}|u_j\rangle \otimes |1\rangle = |0\rangle_c \otimes C\,A^{-1}|b\rangle \otimes |1\rangle$ (Section 1.2). Its squared norm is the probability of measuring 1; dividing by its norm gives $|x\rangle$.

4.4 Eigenvalue inversion and the constant C

The rotation $R_Y(\theta)|0\rangle = \cos\frac{\theta}{2}|0\rangle + \sin\frac{\theta}{2}|1\rangle$ with

$$\theta_k = 2\arcsin\frac{C}{\tilde{\lambda}_k}, \qquad \tilde{\lambda}_k = C\,\tilde{k},$$

puts amplitude $C/\tilde{\lambda}_k$ on $|1\rangle$ — this is how a unitary "divides by λ". Requirements and consequences:

- $|C/\tilde{\lambda}| \le 1$ for every eigenvalue: $C \le |\lambda|_{\min}$. The largest allowed C maximises P_{success}, so the best choice is $C = |\lambda|_{\min}$.
- Negative $\tilde{\lambda}$ gives a negative angle: the sign of $1/\lambda$ is kept in the amplitude.
- In the notebook $C = 2\pi/(2^{n_c}t)$ is the eigenvalue of clock value 1 (the smallest representable $|\tilde{\lambda}|$), and the rotation for register k is $\theta = 2\arcsin(1/\tilde{k})$; with the "auto" time and $\kappa \le 2^{n_c-1}-1$ this means $C = |\lambda|_{\min}$.

Worked example 4.10: rotation angles of the default run

$C = 1$, $\tilde{\lambda} = 1, 2, 3, 4$: $\sin\frac{\theta}{2} = 1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}$, i.e. $\theta = \pi$, $\frac{\pi}{3} = 1.0472$, $2\arcsin\frac{1}{3} = 0.6797$, $2\arcsin\frac{1}{4} = 0.5054$ rad. The ancilla-$|1\rangle$ probabilities for single eigenvectors are $1, \frac{1}{4}, \frac{1}{9}, \frac{1}{16}$. Register 12 means $\tilde{k} = -4$: amplitude $-\frac{1}{4}$.

The notebook's lookup table. For every register value $k = 1, \ldots, 2^{n_c} - 1$ (zero is skipped — no eigenvalue 0): apply `x` to every clock qubit that is 0 in k ("polarity" gates, so that the control fires only on $|k\rangle$), apply $R_Y(\theta_k)$ controlled on *all* n_c clock qubits (`kernel.cry(theta, clock, ancilla)`, recorded as `ccccry` for $n_c = 4$), undo the `x` gates. That is $2^{n_c} - 1$ rotations (15 for $n_c = 4$, 255 for $n_c = 8$) — exponential in n_c. Scalable designs compute $1/\tilde{\lambda}$ with reversible arithmetic or approximate $1/x$ by a polynomial (QSVT, Section 9).

4.5 The ancilla and uncomputation

A^{-1} is not unitary, so it cannot be a gate. HHL embeds it as one block of a larger unitary on system + ancilla: the $|1\rangle$ branch carries $C\,A^{-1}|b\rangle$, the $|0\rangle$ branch the rest. The price is a **probabilistic** outcome.

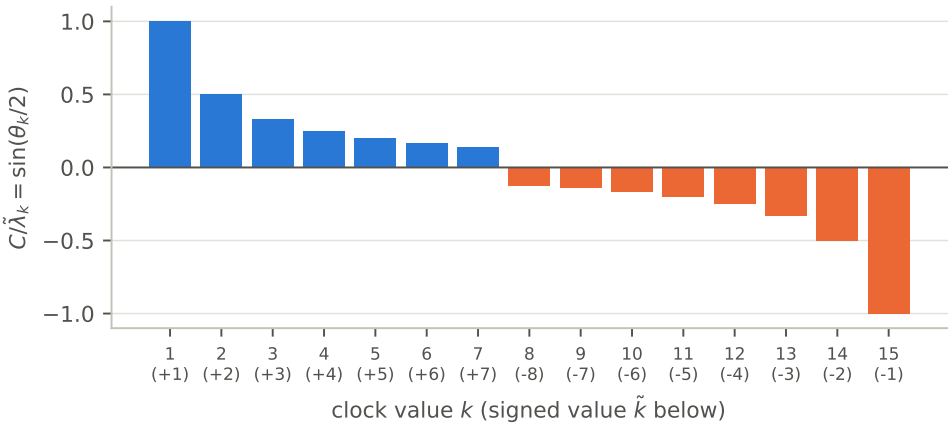


Figure 1: Ancilla amplitude $C/\tilde{\lambda}_k$ for the 15 non-zero clock values of the default run ($n_c = 4$, $C = 1$); register values ≥ 8 are negative.

Before uncomputation the clock is entangled with the system. Post-selecting the ancilla at that point would leave the system **mixed**: the branches $|k_j\rangle$ are orthogonal, so the terms $\frac{\beta_j}{\lambda_j}|u_j\rangle$ cannot interfere into $|x\rangle$. Inverse phase estimation (QFT, controlled U^{-2^j} = `matrix_power(U.conj().T, 2**j)`, $H^{\otimes n_c}$) returns every $|k_j\rangle$ to $|0\rangle$ and disentangles the clock. If some k_j were not integers, the clock would not return exactly to $|0\rangle$; the notebook therefore keeps only shots with clock $= 0$ *and* ancilla $= 1$.

SILICO FELLER

4.6 Success probability and repetitions

Key result: success probability bounds

With $C = |\lambda|_{\min}$: $\frac{1}{\kappa^2} \leq P_{\text{success}} = \sum_j |\beta_j|^2 \frac{C^2}{\lambda_j^2} \leq 1$. Lower bound: all weight on $|\lambda|_{\max}$. Upper bound: all weight on $|\lambda|_{\min}$.

Repeat-until-success needs $1/P_{\text{success}}$ runs on average; **amplitude amplification** needs $O(1/\sqrt{P_{\text{success}}})$ (e.g. $P = 0.01$: about 100 vs. about 10). In the worst case this turns κ^2 repetitions into κ.

Common trap

C/λ is an *amplitude*; the probability is C^2/λ^2. P_{success} is *not* $C\|A^{-1}b\|$, and it does not depend on κ alone but on where b lives.

Checklist

$C \leq |\lambda|_{\min}$? Angles $\theta = 2\arcsin(C/\tilde{\lambda})$ (factor 2!)? Sign of negative eigenvalues kept? Clock uncomputed before reading the system?

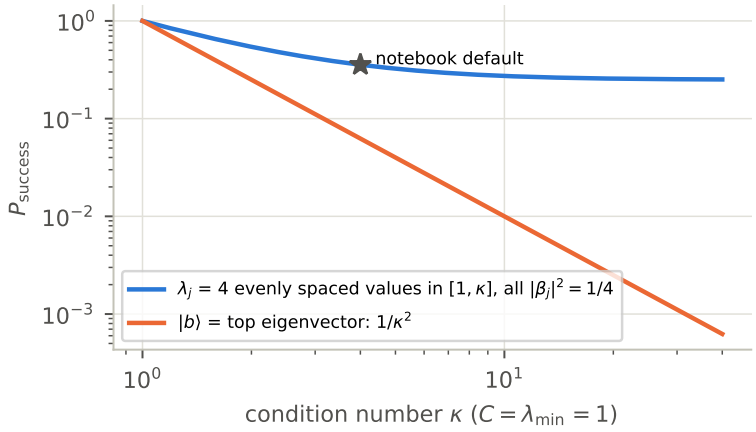


Figure 2: Illustrative: P_{success} versus κ ($C = \lambda_{\min} = 1$) for $|b\rangle$ spread evenly over four eigenvectors with evenly spaced eigenvalues in $[1, \kappa]$, and for $|b\rangle$ equal to the top eigenvector ($1/\kappa^2$). The star is the notebook's default, 0.3559.

5 Fully worked problems

Worked example 5.11: the notebook's 2×2 example

$A = \begin{pmatrix} 1 & -1/3 \\ -1/3 & 1 \end{pmatrix}$, $b = (1, 0)$, $n_c = 4$, "auto" time.

Step 1. $\lambda_1 = \frac{2}{3}$, $|u_1\rangle = \frac{1}{\sqrt{2}}(1, 1)$; $\lambda_2 = \frac{4}{3}$, $|u_2\rangle = \frac{1}{\sqrt{2}}(1, -1)$; $\kappa = 2 \le 7$.

Step 2. $t = \frac{2\pi}{16 \cdot \frac{2}{3}} = \frac{3\pi}{16}$, $C = \lambda_{\min} = \frac{2}{3}$; clock values $k = \lambda/C = 1, 2$ (exact).

Step 3. $\beta_1 = \beta_2 = \frac{1}{\sqrt{2}}$; rotation amplitudes $C/\lambda = 1, \frac{1}{2}$.

Step 4. Ancilla-1 branch: $\frac{1}{\sqrt{2}} \cdot 1 \cdot |u_1\rangle + \frac{1}{\sqrt{2}} \cdot \frac{1}{2} |u_2\rangle = \frac{1}{2}(1, 1) + \frac{1}{4}(1, -1) = (\frac{3}{4}, \frac{1}{4})$.

Step 5. $P_{\text{success}} = \frac{9}{16} + \frac{1}{16} = \frac{5}{8} = 0.625$; $|x\rangle = \frac{1}{\sqrt{10}}(3, 1)$; sampled frequencies 0.9 and 0.1.

Step 6. Norm recovery: $x = \frac{\|b\|}{C}(\frac{3}{4}, \frac{1}{4}) = \frac{3}{2}(\frac{3}{4}, \frac{1}{4}) = (1.125, 0.375)$ ✓. Re-executed with CUDA-Q: $P = 0.625000$, fidelity 1.0000000000.

Worked example 5.12: the notebook's default 4×4 system

Step 1. $\beta_j = \langle u_j | b \rangle = H_{0j} = \frac{1}{2}$ for all four eigenvectors; $t = \pi/8$, $C = 1$, $k_j = \lambda_j = 1, 2, 3, 4$.

Step 2. Ancilla-1 branch: $\sum_j \frac{1}{2} \cdot \frac{1}{\lambda_j} |u_j\rangle = H \cdot \frac{1}{2}(1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4})$:

$$x = \tfrac{1}{4}(1 + \tfrac{1}{2} + \tfrac{1}{3} + \tfrac{1}{4},\ 1 - \tfrac{1}{2} + \tfrac{1}{3} - \tfrac{1}{4},\ 1 + \tfrac{1}{2} - \tfrac{1}{3} - \tfrac{1}{4},\ 1 - \tfrac{1}{2} - \tfrac{1}{3} + \tfrac{1}{4}) = \tfrac{1}{48}(25, 7, 11, 5).$$

Step 3. Decimals: $(0.520833, 0.145833, 0.229167, 0.104167)$ — equal to `np.linalg.solve`. Because $C = \|b\| = 1$, the amplitudes *are* x.

Step 4. $P_{\text{success}} = \|x\|^2 = \frac{625+49+121+25}{2304} = \frac{820}{2304} = \frac{205}{576} = 0.355903$, also $= \frac{1}{4}(1 + \frac{1}{4} + \frac{1}{9} + \frac{1}{16})$. $\|x\| = \sqrt{0.355903} = 0.5966$.

Step 5. Shot probabilities:

$$|x_i|^2 / \|x\|^2 = (625, 49, 121, 25)/820 = (0.7622,\ 0.0598,\ 0.1476,\ 0.0305).$$

Step 6. With 20 000 shots, about $0.355903 \times 20\,000 \approx 7118$ are accepted (recorded: 7086).

Worked example 5.13: a diagonal system with three clock qubits

$A = \mathrm{diag}(1, 2)$, $|b\rangle = \frac{1}{\sqrt{2}}(1, 1)$, $n_c = 3$, $t = \pi/4$.

Step 1. $C = \frac{2\pi}{8 \cdot \pi/4} = 1$; clock values $k = \lambda$: $1 = \texttt{001}$ and $2 = \texttt{010}$.

Step 2. After phase estimation: $\frac{1}{\sqrt{2}}(|0\rangle\,|001\rangle + |1\rangle\,|010\rangle)$ (system $\otimes$ clock).

Step 3. Ancilla-1 branch: $\frac{1}{\sqrt{2}}(1 \cdot |0\rangle + \frac{1}{2}\,|1\rangle)$; $P_{\text{success}} = \frac{1}{2}(1 + \frac{1}{4}) = \frac{5}{8} = 0.625$.

Step 4. $|x\rangle \propto (1, \frac{1}{2})$, normalised $\frac{1}{\sqrt{5}}(2, 1)$ — matches $A^{-1}b \propto (1, \frac{1}{2})$.

Worked example 5.14: a negative eigenvalue

$A = \mathrm{diag}(1, -2)$, $|b\rangle = \frac{1}{\sqrt{2}}(1, 1)$, $n_c = 3$: $\kappa = 2 \le 3$, so $t = \frac{2\pi}{8 \cdot 1} = \frac{\pi}{4}$, $C = 1$.

Step 1. Clock values: $\lambda = 1 \to 1$; $\lambda = -2 \to -2 \to 8 - 2 = 6 = \texttt{110}$.

Step 2. The lookup table reads register 6 as $\tilde{k} = -2$: amplitude $C/\tilde{\lambda} = -\frac{1}{2}$ (negative angle).

Step 3. Ancilla-1 branch: $\frac{1}{\sqrt{2}}(1, -\frac{1}{2})$, $P_{\text{success}} = 0.625$; $A^{-1}b = \frac{1}{\sqrt{2}}(1, -\frac{1}{2})$ ✓(sign preserved).

Worked example 5.15: too few clock qubits (re-executed variant)

Default system with $n_c = 3$: $k_{\max} = 3 < \kappa = 4$, so "auto" maps $\lambda_{\max} = 4$ to 3: $t = \frac{2\pi \cdot 3}{8 \cdot 4} = \frac{3\pi}{16}$, $C = \frac{4}{3}$.

Step 1. Clock values $\lambda \cdot \frac{3}{4} = 0.75, 1.5, 2.25, 3$: only $\lambda = 4$ is exact; the others leak over neighbouring registers (clock distribution: $P(1) = P(2) = 0.3147$, $P(3) = 0.2891$).

Step 2. Re-executed with CUDA-Q: fidelity 0.9532157640, $P_{\text{success}} = 0.3285$ from the state.

Step 3. The notebook's printed "ideal" value $\sum_j |\beta_j|^2 C^2/\lambda_j^2 = 0.632716$ uses $C/\lambda_1 = \frac{4}{3} > 1$ — not a probability. The formula is valid only when $C \le |\lambda|_{\min}$ (here it is not); the circuit itself clips nothing, because every register value has $|\tilde{k}| \ge 1$.

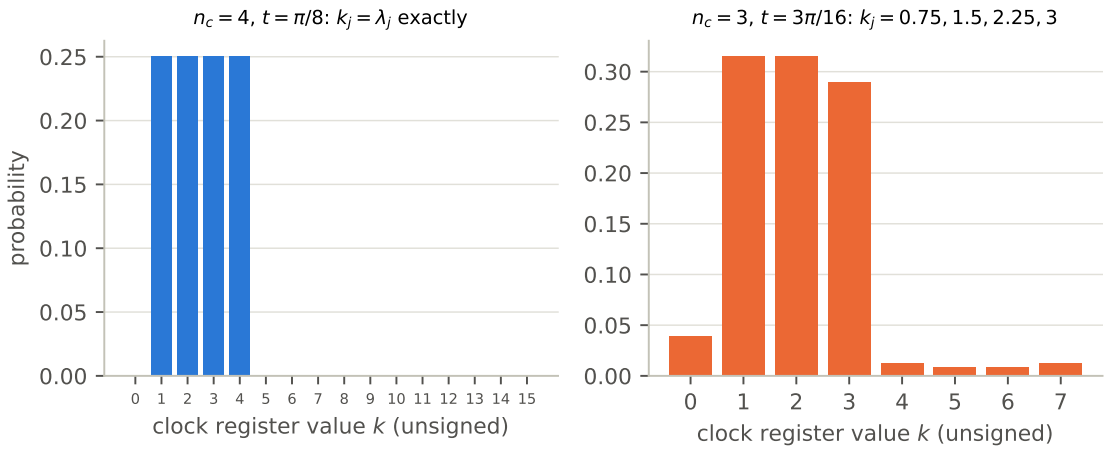


Figure 3: Clock register after phase estimation for the default system (CUDA-Q, notebook circuit stopped after the inverse QFT). Left: $n_c = 4$, four exact peaks of $\frac{1}{4}$. Right: $n_c = 3$, rounded clock values spread over neighbouring registers.

6 Reading out the solution

Key result: what $|x\rangle$ gives you

- **Samples**: outcome i with probability $|x_i|^2/\|x\|^2$ (magnitudes only).
- **Expectation values** $\langle x|M|x\rangle$ of an observable M — the intended use.
- **Overlaps** with another state via a swap test: $P(0) = \frac{1}{2}(1 + |\langle x|\phi\rangle|^2)$.
- **The norm**: $\|x\| = \frac{\|b\|}{C}\sqrt{P_{\text{success}}}$, so the success rate estimates $\|A^{-1}b\|$.

Estimating all d components to precision ε costs about d/ε^2 shots — linear in d, no speed-up.

Worked example 6.16: statistics of the recorded run

Step 1. Accepted fraction: $7086/20\,000 = 0.3543$; its binomial standard error $\sqrt{p(1-p)/S} = \sqrt{0.3559 \cdot 0.6441/20\,000} = 0.0034$. The deviation 0.0016 is about half a standard error.

Step 2. Component 0: sampled 0.7691 vs. exact 0.7622 (difference 0.0069); standard error $\sqrt{0.7622 \cdot 0.2378/7086} = 0.0051$, so 1.4 standard errors — ordinary shot noise. The notebook quotes the conservative scale $1/\sqrt{7086} = 0.012$.

Step 3. Norm from the success rate ($C = \|b\| = 1$): $\|x\| \approx \sqrt{0.3543} = 0.595$ (exact 0.5966).

Step 4. An expectation value: Z on the first system qubit (the most significant bit) gives $\langle Z \otimes I\rangle = \frac{625+49-121-25}{820} = 0.6439$.

Signs and phases are invisible in computational-basis samples: $x = (1, -1)$ and $(1, 1)$ give the same histogram. They need measurements in other bases or interference with a reference state. **In the notebook** the simulator reads the amplitudes directly (`cudaq.get_state`), takes the branch clock = 0, ancilla = 1, and multiplies by $\|b\|/C$ — full amplitude access that hardware does not provide.

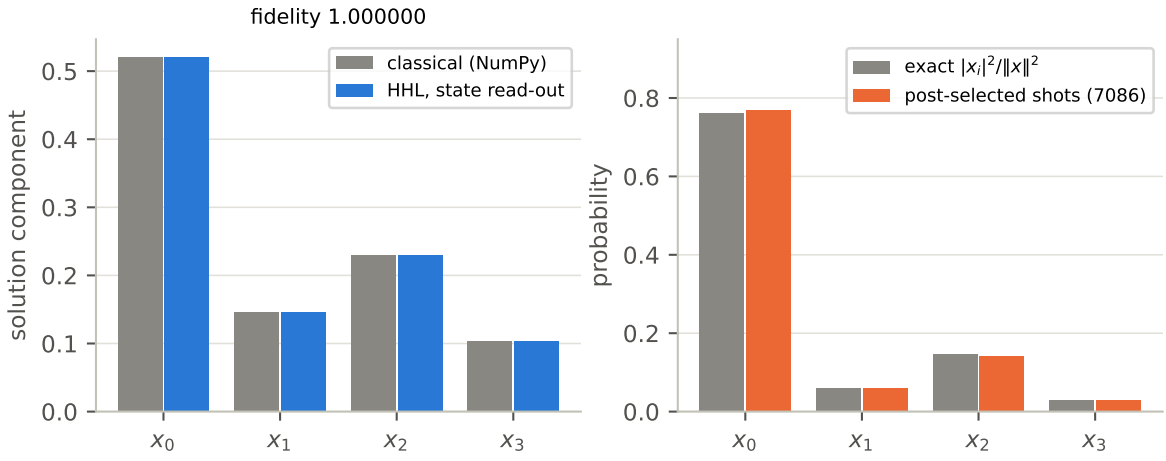


Figure 4: Default run, re-executed with the notebook's circuit and seed 42 (identical to the recorded output). Left: state read-out vs. NumPy. Right: the 7086 post-selected shots vs. the exact $|x_i|^2/\|x\|^2$.

Common trap

"HHL outputs the vector x exponentially faster" — it outputs a *state*. Printing x needs $O(d/\varepsilon^2)$ repetitions. And "the norm is lost" is also wrong: the success probability carries $\|A^{-1}b\|$.

7 Complexity and its fine print

Key result: HHL running time (Harrow, Hassidim, Lloyd 2009)

$\tilde{O}(\log(d)\, s^2\kappa^2/\varepsilon)$ for an s-sparse $d \times d$ matrix, condition number κ and precision ε. $\log d$: amplitude encoding and sparse Hamiltonian simulation; one κ from the phase-estimation precision needed to resolve $\lambda_{\min}$, one from the success probability (after amplitude amplification); $1/\varepsilon$ from phase estimation.

Compared with conjugate gradient ($O(d\, s\sqrt{\kappa}\log(1/\varepsilon))$ for positive-definite A), HHL is exponentially better in d but worse in κ and ε. The speed-up survives only under all of the following (Aaronson, "Read the fine print", Nature Physics 2015):

1. $|b\rangle$ can be prepared in polylog(d) time;
2. e^{iAt} can be applied efficiently (A sparse or structured, accessed through an oracle);
3. κ (and $1/\varepsilon$) grow at most polylogarithmically in d;
4. only a global property of x is needed (an expectation value, an overlap, a norm).

SELCO FELLER

Evidence on both sides. HHL is BQP-complete: simulating all its instances classically would simulate any quantum computation, so in general no classical algorithm matches it. On the other hand, "dequantised" algorithms (Tang 2018 and later) show that for *low-rank* matrices with classical sampling access, classical algorithms match the quantum scaling up to polynomial factors — some proposed quantum advantages disappear.

Worked example 7.17: counting clock qubits for a given κ

The notebook needs $\kappa \le 2^{n_c-1} - 1$ for exact clock values with "auto" time: $\kappa = 3 \to n_c = 3$; $\kappa = 7 \to 4$; $\kappa = 20 \to 6$ ($2^5 - 1 = 31 \ge 20$, while $2^4 - 1 = 15 < 20$); $\kappa = 40 \to 7$. The circuit then has $2^{n_c} - 1$ rotations: $7, 15, 63, 127$.

Worked example 7.18: the worst-case success probability

$\kappa = 20$, $C = \lambda_{\min}$, $|b\rangle$ = eigenvector of $\lambda_{\max}$: $P_{\text{success}} = (\lambda_{\min}/\lambda_{\max})^2 = 1/400 = 0.0025$, i.e. 400 repetitions on average (about 20 with amplitude amplification).

Common trap

"HHL gives an exponential speed-up for every linear system" — false: dense A, large κ, expensive $|b\rangle$ or needing all of x each remove it.

8 The implementation in HHL.ipynb

8.1 Structure and parameters

1. Section 3 (inputs): `MATRIX_A = "example"` (or `"random"` or a list), `VECTOR_B = "example"`, `RANDOM_SIZE = 4`, `RANDOM_CONDITION_NUMBER = 4.0`, `NUM_CLOCK_QUBITS = 4`, `EVOLUTION_TIME = "auto"`, `BACKEND = "auto"`, `SIMULATION_METHOD = "statevector"` (allowed: `"statevector"`, `"density_matrix"`), `SHOTS = 20000`, `SEED = 42`.
2. Section 4: build and validate (square A, matching b, $b \neq 0$, $|\det A| \geq 10^{-12}$ and $\mathrm{cond}(A) \leq 10^{12}$), `x_classical = np.linalg.solve(...)`, Hermitian embedding, identity padding, eigenvalues, κ, $n_c \geq 2$, the time t, C and the qubit count.
3. Section 6: expected results *before* running: $|x\rangle$, the "ideal" P_{success}, whether the mapping is exact, memory, a gate estimate.
4. Section 7: `build_hhl_kernel` with the kernel builder (`cudaq.make_kernel`), drawing, `cudaq.estimate_resources` statistics.
5. Section 8: `cudaq.get_state` (amplitude read-out) and `cudaq.sample` (shots), with live memory monitoring.
6. Sections 9–10: tables, plots, benchmark report in `exports/hhl/`.

Key result: the "auto" evolution time

With $k_{\max} = 2^{n_c-1} - 1$: if $\kappa \leq k_{\max}$, $t = \dfrac{2\pi}{2^{n_c}|\lambda|_{\min}}$ ($|\lambda|_{\min} \to$ clock value 1, $C = |\lambda|_{\min}$);
otherwise $t = \dfrac{2\pi k_{\max}}{2^{n_c}|\lambda|_{\max}}$ ($|\lambda|_{\max} \to k_{\max}$; small eigenvalues are rounded and $C > |\lambda|_{\min}$).

8.2 The circuit code

The builder allocates one register initialised with $|0\rangle_c \otimes |b\rangle \otimes |0\rangle_a$ (bit-reversed, because CUDA-Q's state vector treats qubit 0 as the least significant bit), then:

1. `h` on every clock qubit; for $j = 0, \ldots, n_c - 1$ a custom gate `matrix_power(U, 2**j)` called as `kernel.<name>(clock[n_c-1-j], *system)` (first argument = control; `clock[0]` is the MSB);
2. inverse QFT (`swap`, `cr1`$(-\pi/2^{k-j})$, `h`);
3. for $k = 1, \ldots, 2^{n_c} - 1$: `signed = k - 2**nc if k >= 2**(nc-1) else k`, `theta = 2*arcsin(clip(C/lam_k, -1, 1))`, polarity `x` gates, `kernel.cry(theta, clock, ancilla)`, polarity `x` gates;
4. QFT, `swaps`, `matrix_power(U.conj().T, 2**j)`, `h`;
5. optionally `mz(reg)`: the bitstring is clock | system | ancilla, qubit 0 first.

Read-out: `postselected_amplitudes` reshapes the state into $2 \times \cdots \times 2$, reverses the axes and takes index $(0, \ldots, 0, :, \ldots, :, 1)$; `x_quantum = amplitudes * b_norm / C_constant`, then the embedding/padding slice. Shots are kept if `bits[:n_c] == "0"*n_c` and `bits[-1] == "1"`. With `"density_matrix"` the dominant eigenvector of ρ is used and its arbitrary global phase is aligned with NumPy's solution (a density matrix carries no global phase). Hardware backends are refused with a `ValueError`: exact matrix exponentials as custom gates and amplitude loading cannot run on a QPU.

8.3 Circuit size

Counting the builder's gates for n_c clock qubits ($n_b = 2$):

$$\underbrace{4n_c}_{\texttt{h}} + \underbrace{2n_c}_{\text{custom}} + \underbrace{2\lfloor n_c/2 \rfloor}_{\texttt{swap}} + \underbrace{n_c(n_c-1)}_{\texttt{cr1}} + \underbrace{2\left(n_c 2^{n_c-1} - n_c\right)}_{\texttt{x}} + \underbrace{2^{n_c}-1}_{\text{rotations}}.$$

(The `x` count: register values $1, \ldots, 2^{n_c}-1$ contain $n_c 2^{n_c-1} - n_c$ zero bits in total, each flipped twice.)

n_c	qubits	gates	rotations	depth	multi-qubit
2	5	23	3	21	11
3	6	51	7	38	21
4	7	111	15	67	39
5	8	235	31	120	65
6	9	507	63	221	111
7	10	1099	127	418	189

From `cudaq.estimate_resources` on the notebook's builder (4×4 system, "auto" time, measurements and the amplitude loading not counted). The recorded default ($n_c = 4$): `h` 16, custom 8, `swap` 4, `cr1` 12, `x` 56, `ccccry` 15 = 111 gates; 39 multi-qubit; depth 67. The notebook's pre-run estimate of "∼107 gates" uses a simpler formula (it counts $1 + n_c$ gates per rotation instead of the actual `x` gates) — an estimate, not a count.

8.4 Results

run	settings	C	P_{success} (state)	fidelity	accepted shots
recorded	default, $n_c = 4$	1	0.355903	1.0000000000	7086 / 20000
re-executed	default, $n_c = 4$	1	0.355903	1.0000000000	7086 / 20000
re-executed variant	default, $n_c = 3$	$\frac{4}{3}$	0.3285	0.9532157640	—
re-executed variant	default, $n_c = 2$	4	0.4222	0.2059	—
re-executed variant	2×2 example	$\frac{2}{3}$	0.625000	1.0000000000	—

Relative error of the recovered x: 1.655×10^{-15} (recorded), 1.74×10^{-15} (re-executed) — floating-point level. Recorded timings on a CPU (`qpp-cpu`): construction 0.0056 s, compilation 0.0524 s, state run 0.0018 s, sampling run 0.0758 s. No GPU timing was measured for this material.

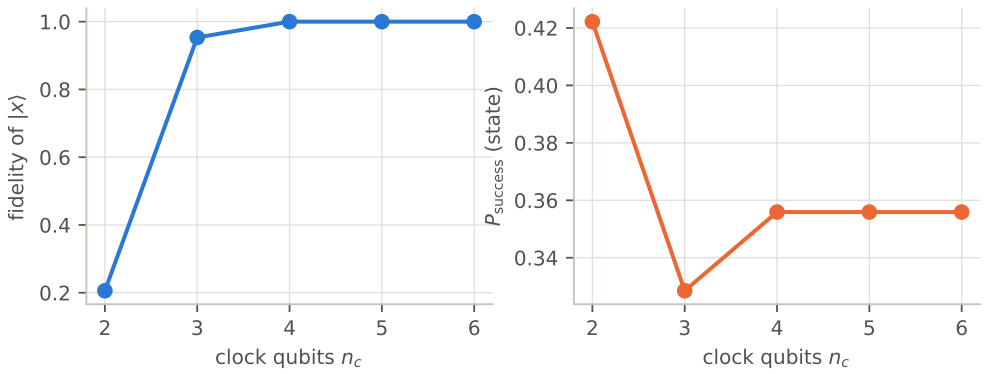


Figure 5: Fidelity and success probability of the default system versus n_c ("auto" time, CUDA-Q state read-out, re-executed). From $n_c = 4$ every clock value is an integer.

Common trap

A high P_{success} is not a good result: at $n_c = 2$ it is 0.42 while the fidelity is 0.21. Always check the fidelity (or the relative error) as well.

Checklist

Recorded, re-executed or illustrative number? Which t and C did "auto" pick? Embedded/-padded system sliced back? Hardware claims — none.

9 Noise, limitations and advanced algorithms

9.1 Errors

Algorithmic (present without noise): phase-estimation rounding when $k_j \notin \mathbb{Z}$ (Section 5), Hamiltonian-simulation error (absent in the notebook: exact `expm`), finite shots ($\sim 1/\sqrt{\text{accepted}}$). **Hardware noise** (general statements; not simulated in the notebook, which defines no noise model even with `"density_matrix"`): the multi-controlled rotations and controlled e^{iAt2^j} decompose into many two-qubit gates; errors in the clock spoil the uncomputation, and post-selection then accepts wrong shots. Practical HHL needs fault-tolerant hardware.

9.2 Limitations

General: state preparation of a generic $|b\rangle$ costs $O(d)$; e^{iAt} must be efficient; the cost grows like κ^2 (or κ); the output is a state; the circuits are deep. Notebook-specific: exact `expm` gates, amplitude loading by `qalloc(vector)`, a lookup-table inversion with $2^{n_c} - 1$ rotations, full-matrix storage (sparsity not used), and simulator-only amplitude read-out.

9.3 Applications and newer algorithms

Proposed applications: discretised differential equations, least squares and other machine-learning kernels, electromagnetic scattering, network quantities such as effective resistances — always for a *global property* of x, and none with a demonstrated end-to-end advantage.

algorithm	key idea
HHL (2009)	phase estimation + eigenvalue-controlled rotation; $\tilde{O}(\log(d)s^2\kappa^2/\varepsilon)$
Ambainis (2012)	variable-time amplitude amplification: $\kappa^2 \to \kappa$ (up to logarithms)
Childs–Kothari–Somma (2017)	linear combination of unitaries (Fourier/Chebyshev series of $1/x$) instead of phase estimation: $\operatorname{poly}\log(1/\varepsilon)$
QSVT (Gilyén et al. 2019)	polynomial approximation of $1/x$ applied to a block-encoding; unifies the above
Adiabatic and randomised methods (Subaşı–Somma–Orsucci 2019), discrete adiabatic (Costa et al. 2022)	optimal $O(\kappa \log(1/\varepsilon))$ query complexity
Variational (VQLS)	short circuits and a classical optimiser; heuristic, no guarantee

None of these is implemented in `HHL.ipynb`; all share HHL's input and output caveats.

10 Problem-solving strategies and question map

10.1 Strategy by question format

- **Single-correct numerical:** compute fully, then sanity-check: $P_{\text{success}} \in [1/\kappa^2, 1]$; $|C/\tilde{\lambda}| \leq 1$; probabilities are squared amplitudes; clock values are λ/C, signed for $k \geq 2^{n_c-1}$. Typical distractors: "amplitude instead of probability", "θ without the factor 2", "phase instead of clock value", "first half of the embedded solution", "forgot the square root" (norm from P).
- **Matching lists:** solve the easiest row first (often a definition or a one-line computation), then eliminate.
- **Assertion–Reason:** judge A and R *independently*; only if both are true ask whether R *explains* A.
- **Statement I/II:** one counterexample kills a "for every" statement; read code statements against the actual code (Section 8).
- **Paragraph:** the first question sets up a quantity (eigenvalues, β_j, the state after QPE) reused in the second.

10.2 Fast checks that catch most mistakes

1. Verify a candidate solution by multiplying: $Ax = b$?
2. With $C = 1$ and $\|b\| = 1$: $P_{\text{success}} = \|x\|^2$.
3. Clock value $= \lambda t 2^{n_c}/(2\pi)$; integer $\Rightarrow$ exact, otherwise leakage and fidelity < 1.
4. Gate counts: rotations $2^{n_c} - 1$; qubits $n_c + n_b + 1$; state vector $16 \cdot 2^N$ bytes.
5. Recorded vs. re-executed vs. illustrative — which one does the question use?

10.3 Map: assessment questions → sections of this chapter

Questions	Where to learn it
Q.1–Q.4, Q.7, Q.8, Q.19, Q.20	Section 1 (systems, eigen-decomposition, matrix classes, condition number)
Q.6, Q.66, Q.67	Section 1.3 and Section 8 (embedding and padding)
Q.9–Q.11	Section 1.5 (sparsity, classical cost)
Q.12, Q.15–Q.17, Q.33, Q.34, Q.76	Section 2 (encoding, e^{iAt}, memory)
Q.21–Q.26, Q.29, Q.30, Q.37–Q.40, Q.71	Section 3 (controlled unitaries, QFT, phase estimation, signed clock values, aliasing)
Q.27, Q.28, Q.31, Q.32, Q.35, Q.41–Q.44, Q.48, Q.50–Q.52	Section 4 (registers, inversion, C, ancilla, uncomputation)
Q.5, Q.13, Q.14, Q.18, Q.45, Q.53, Q.59, Q.60	Section 5 (worked 2×2 and 4×4 problems)
Q.36, Q.54–Q.56, Q.62, Q.63, Q.79, Q.80	Section 6 (read-out, shots, norm, swap test)
Q.61, Q.64, Q.68–Q.70, Q.78	Section 7 (complexity, fine print, dequantisation)
Q.46, Q.47, Q.49, Q.57, Q.58, Q.65, Q.73–Q.75, Q.77	Section 8 (the notebook: auto time, code, statistics, results)
Q.72	Section 9 (advanced algorithms)

(Section numbers: 1 linear algebra, 2 encoding, 3 phase estimation, 4 HHL algorithm, 5 worked problems, 6 read-out, 7 complexity, 8 notebook, 9 noise, limitations and advanced algorithms.)

SILICO FEILER

11 Practice problems

Try each before looking at the answers on the next page. They use the same ideas as the assessment but different numbers.

P1. Condition number of $\mathrm{diag}(4, 0.5)$.

P2. Solve $\left(\begin{smallmatrix} 3 & 1 \\ 1 & 3 \end{smallmatrix}\right)x = (4, 4)$ through the eigen-decomposition.

P3. Amplitude-encode $b = (2, 0, 1, 2)$: $|b\rangle$ and the probability of $|11\rangle$.

P4. e^{iAt} for $A = \mathrm{diag}(1, 3)$ and $t = \pi/2$.

P5. Phase estimation with 3 clock qubits on an eigenstate with $\phi = \frac{3}{8}$: outcome and probability.

P6. With $n_c = 3$, which signed value does register 5 represent?

P7. $n_c = 5$ and $C = 0.5$: the clock value of $\lambda = 3$.

P8. $C = 0.5$, $\lambda = 2$: rotation angle and probability of ancilla $= 1$ for that eigenvector.

P9. $A = \mathrm{diag}(1, 4)$, $|b\rangle = \frac{1}{\sqrt{2}}(1, 1)$, $C = 1$: P_{success} and $|x\rangle$.

P10. $A = \mathrm{diag}(2, -1)$, $n_c = 3$, "auto" time: t, C and the two register values.

P11. A run succeeds with probability 0.04. Expected repetitions with and without amplitude amplification.

P12. Qubits and rotations of the notebook's circuit for a 16×16 Hermitian system with $n_c = 6$.

P13. Use the gate-count formula of Section 8.3 for $n_c = 3$.

P14. Standard error of the accepted fraction for $p = 0.25$ and $S = 10\,000$ shots.

P15. $C = 1$, $\|b\| = 2$, measured success fraction 0.09: estimate $\|x\|$.

P16. A run prints an "ideal" $P_{\text{success}} = 0.63$ while the state gives 0.33 and the fidelity is 0.95. What happened?

SEICO FELLER

Answers to the practice problems

P1. $4/0.5 = 8$.

P2. $b = (4,4) = 4\sqrt{2}\,|u_1\rangle$ with $\lambda_1 = 4$, $|u_1\rangle = \frac{1}{\sqrt{2}}(1,1)$: $x = \frac{4\sqrt{2}}{4}\,|u_1\rangle = (1,1)$. Check: $A(1,1) = (4,4)$.

P3. $\|b\| = 3$: $|b\rangle = \frac{1}{3}(2\,|00\rangle + |10\rangle + 2\,|11\rangle)$; $P(|11\rangle) = \frac{4}{9}$.

P4. $\mathrm{diag}(e^{i\pi/2}, e^{3i\pi/2}) = \mathrm{diag}(i, -i)$.

P5. $8\phi = 3$: $|011\rangle$ with probability 1.

P6. $5 - 8 = -3$.

P7. $k = \lambda/C = 6$ (`00110`).

P8. $\theta = 2\arcsin\frac{0.5}{2} = 2\arcsin\frac{1}{4} = 0.5054$ rad; $P = \frac{1}{16} = 0.0625$.

P9. $P = \frac{1}{2}(1 + \frac{1}{16}) = \frac{17}{32} = 0.53125$; $|x\rangle \propto (1, \frac{1}{4})$, i.e. $\frac{1}{\sqrt{17}}(4,1)$.

P10. $\kappa = 2 \le 3$: $t = \frac{2\pi}{8\cdot 1} = \frac{\pi}{4}$, $C = 1$; $\lambda = 2 \to 2$ (`010`), $\lambda = -1 \to 8 - 1 = 7$ (`111`).

P11. $1/0.04 = 25$ without; about $1/\sqrt{0.04} = 5$ with amplitude amplification.

P12. $6 + 4 + 1 = 11$ qubits; $2^6 - 1 = 63$ rotations.

P13. $12 + 6 + 2 + 6 + 2(12 - 3) + 7 = 51$ (matches `estimate_resources`).

P14. $\sqrt{0.25 \cdot 0.75/10\,000} = 0.0043$.

P15. $\|x\| = \frac{\|b\|}{C}\sqrt{P} = 2 \times 0.3 = 0.6$.

P16. κ exceeded $2^{n_c-1} - 1$, so "auto" set $C > |\lambda|_{\min}$: the "ideal" formula used $C/\lambda > 1$ and is not a probability, and the eigenvalues were rounded (fidelity < 1). Increase `NUM_CLOCK_QUBITS` (this is exactly the $n_c = 3$ case of the default system).

Summary of the chapter in ten lines

1. $A^{-1}\,|b\rangle = \sum_j \frac{\beta_j}{\lambda_j}\,|u_j\rangle$: inversion is division in the eigenbasis of a Hermitian (or embedded) A.
2. $\kappa = |\lambda|_{\max}/|\lambda|_{\min}$ sets clock size, success probability and cost; padding can enlarge it.
3. Amplitude encoding stores a direction in $\log_2 d$ qubits; preparing and reading it are the expensive ends.
4. Phase estimation with $U = e^{iAt}$ writes $k_j = \lambda_j/C$, $C = 2\pi/(2^{n_c}t)$; negative values in two's complement; non-integers leak.
5. $R_Y(2\arcsin(C/\tilde{\lambda}))$ puts C/λ_j on the ancilla; $C \le |\lambda|_{\min}$.
6. Uncompute the clock, post-select ancilla = 1: $|x\rangle$ with $P_{\text{success}} = C^2\|A^{-1}\,|b\rangle\,\|^2 \in [1/\kappa^2, 1]$.
7. Read-out: samples, expectation values, overlaps and the norm — not the full vector (d/ε^2 shots).
8. Cost $\tilde{O}(\log(d)s^2\kappa^2/\varepsilon)$, valid only under the input/output fine print; newer algorithms reach $O(\kappa\log(1/\varepsilon))$.
9. `HHL.ipynb`: exact `expm` gates, lookup-table inversion, 7 qubits, 111 gates; $x = (25, 7, 11, 5)/48$ with fidelity 1 and $P = 0.3559$ — reproduced with CUDA-Q.
10. Report honestly: recorded vs. re-executed vs. illustrative; check fidelity, not only P_{success}; no hardware or GPU claims without measurements.